Ananya Yallapragada
019142383

Homework 8 Report

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	2383
PORT_BASE	8583
PREFIX	s2383
SEED	2383
VERIFY_SEED	262383
DOMAIN_ID	7
Hardware	MacBook Pro, Apple M4 (10-core CPU), 16 GB RAM
Python version	3.11.9
Tagged commit hash	

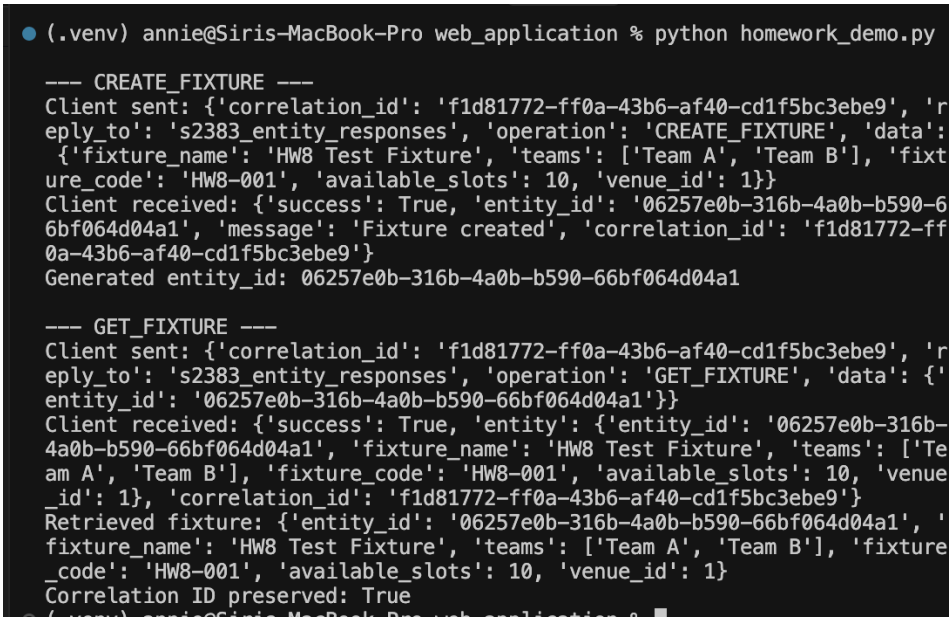
Github -
(Please note that since the commit hash needs to be included in the report, a new commit will be created when the report is added again with the hash included. The tag link here is before the hash was added. The updated tag is pasted in the comments on canvas.).

Ananya Yallapragada
019142383

Part 1 - Backend Service Request/Response over Kafka

I. Operations, II. Kafka Requirements and IV. Demonstration

client



worker



Ananya Yallapragada
019142383

II. Kafka Requirements (in prev section)

III. General Requirements

Validation:

1. Worker terminal

```
(base) annie@Siris-MacBook-Pro web_application % cd /Users/annie/Library/CloudStorage/OneDrive-Personal/Academics/Mas
ters/SJSU/DATA-260/Homework/data260-2383/code/web_application
../../.venv/bin/python -m kafka.worker
eration': 'CREATE_FIXTURE', 'data': {'fixture_name': '   ', 'teams': ['Team A', 'Team B'], 'fixture_code': 'VALID-001
', 'available_slots': 10, 'venue_id': 1}}
Worker processed: {'success': False, 'error': 'fixture_name cannot be empty', 'correlation_id': '5034f583-5cbd-4e17-b
ae0-2790c623fdac'}
Worker sent response: {'success': False, 'error': 'fixture_name cannot be empty', 'correlation_id': '5034f583-5cbd-4e
17-bae0-2790c623fdac'}
Worker received: {'correlation_id': 'db3a5785-2122-4ee9-88ec-61aaf883edb0', 'reply_to': 's2383_entity_responses', 'op
eration': 'CREATE_FIXTURE', 'data': {'fixture_name': 'Validation Fixture', 'teams': [], 'fixture_code': 'VALID-001',
'available_slots': 10, 'venue_id': 1}}
Worker processed: {'success': False, 'error': 'teams must be a non-empty list', 'correlation_id': 'db3a5785-2122-4ee9
-88ec-61aaf883edb0'}
Worker sent response: {'success': False, 'error': 'teams must be a non-empty list', 'correlation_id': 'db3a5785-2122-
4ee9-88ec-61aaf883edb0'}
Worker received: {'correlation_id': '7f92f420-d58f-4777-9bb3-2dcd1efd9c70', 'reply_to': 's2383_entity_responses', 'op
eration': 'CREATE_FIXTURE', 'data': {'fixture_name': 'Validation Fixture', 'teams': ['Team A', 'Team B'], 'fixture_co
de': '', 'available_slots': 10, 'venue_id': 1}}
Worker processed: {'success': False, 'error': 'fixture_code cannot be empty', 'correlation_id': '7f92f420-d58f-4777-9
bb3-2dcd1efd9c70'}
Worker sent response: {'success': False, 'error': 'fixture_code cannot be empty', 'correlation_id': '7f92f420-d58f-47
77-9bb3-2dcd1efd9c70'}
```

2. Validation terminal

```
(base) annie@mac web_application % clear
(base) annie@mac web_application % cd /Users/annie/Library/CloudStorage/OneDrive-Personal/Academics/Masters/SJSU/DATA
-260/Homework/data260-2383/code/web_application
../../.venv/bin/python validate_kafka_errors.py
Student: Ananya
HW8 Part 1 Kafka validation/error test
Client sent: {'correlation_id': '41edb7db-0bcf-4cde-9f11-8a9628329bd6', 'reply_to': 's2383_entity_responses', 'operat
ion': 'CREATE_FIXTURE', 'data': {'fixture_name': 'Validation Fixture', 'teams': ['Team A', 'Team B'], 'available_slot
s': 10, 'venue_id': 1}}
Client received: {'success': False, 'error': 'Missing required field: fixture_code', 'correlation_id': '41edb7db-0bcf
-4cde-9f11-8a9628329bd6'}
Missing fixture_code: PASS
Response: {'success': False, 'error': 'Missing required field: fixture_code', 'correlation_id': '41edb7db-0bcf-4cde-9
f11-8a9628329bd6'}
```

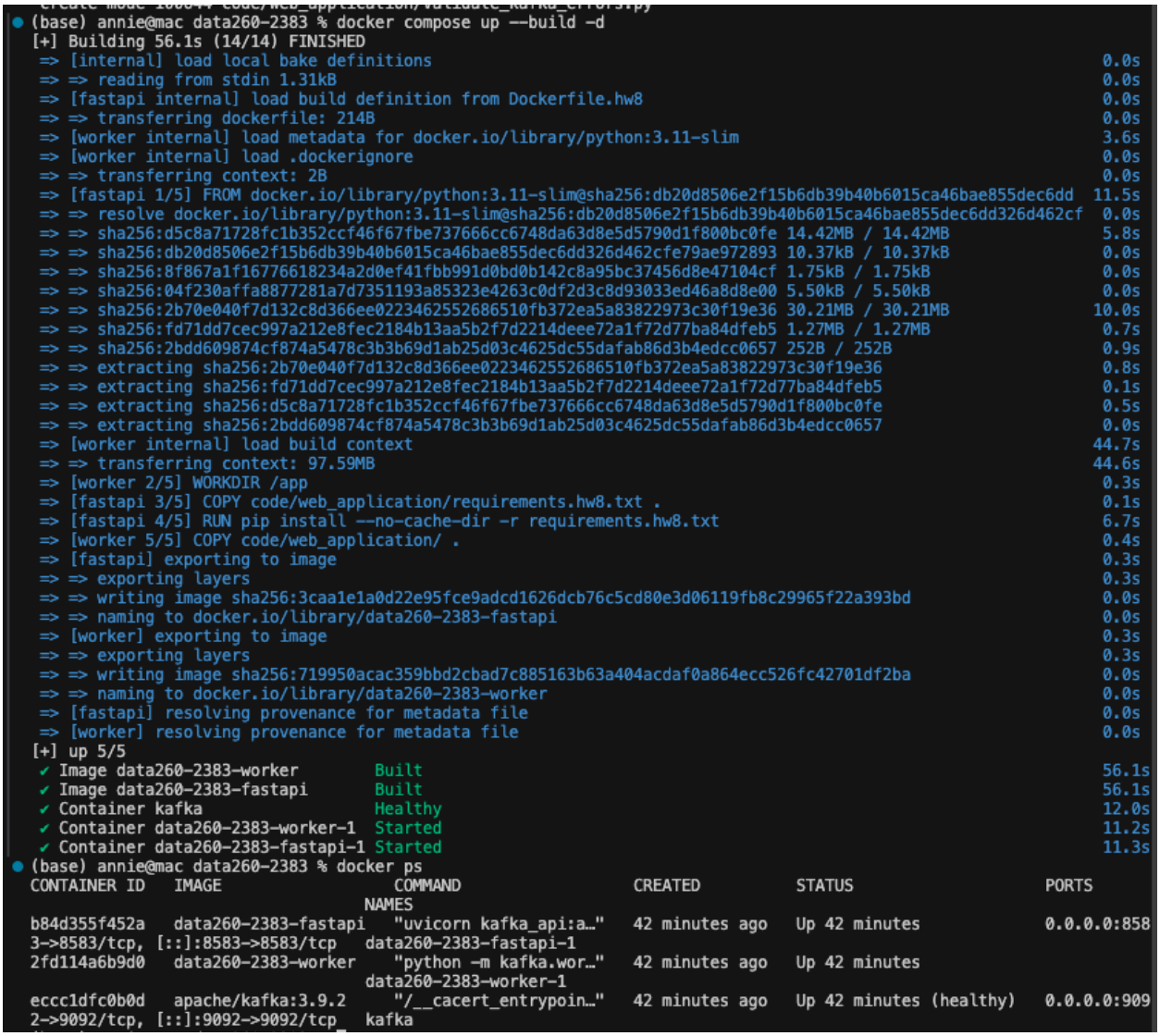
```
GET unknown entity_id: PASS
Response: {'success': False, 'error': 'Fixture not found', 'correlation_id': 'b9564917-1f78-47f6-9381-98e6b90f4e50'}
All validation/error cases passed.
```

IV. Demonstration in Section I.

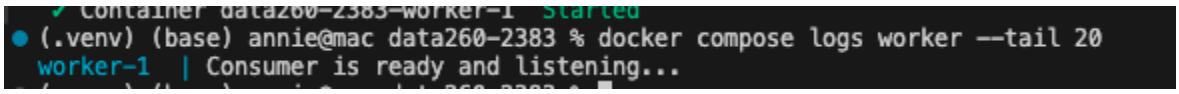
Ananya Yallapragada
019142383

V. Required Screenshots and Evidence.

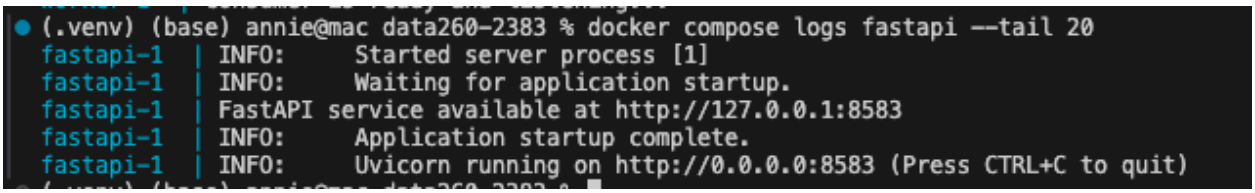
docker ps



worker log



fast-api startup log



Ananya Yallapragada
019142383

Successful demo:

```
(.venv) annie@mac web_application % python homework_demo.py

--- CREATE_FIXTURE ---
Client sent: {'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc', 'reply_to': 's2383_entity_responses', 'operat
ion': 'CREATE_FIXTURE', 'data': {'fixture_name': 'HW8 Test Fixture', 'teams': ['Team A', 'Team B'], 'fixture_code': '
HW8-001', 'available_slots': 10, 'venue_id': 1}}
Client received: {'success': True, 'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'message': 'Fixture created',
 'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc'}
Generated entity_id: 1d480354-a6d8-436c-9e2e-d6ef142e6685

--- GET_FIXTURE ---
Client sent: {'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc', 'reply_to': 's2383_entity_responses', 'operat
ion': 'GET_FIXTURE', 'data': {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685'}}
Client received: {'success': True, 'entity': {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'fixture_name': 'H
W8 Test Fixture', 'teams': ['Team A', 'Team B'], 'fixture_code': 'HW8-001', 'available_slots': 10, 'venue_id': 1}, 'c
orrelation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc'}
Retrieved fixture: {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'fixture_name': 'HW8 Test Fixture', 'teams':
 ['Team A', 'Team B'], 'fixture_code': 'HW8-001', 'available_slots': 10, 'venue_id': 1}
Correlation ID preserved: True
```

Part I relevant code snippets:
kafka/client.py - Required topic names and request schema

(.venv) annie@mac web_application % python homework_demo.py

--- CREATE_FIXTURE ---
Client sent: {'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc', 'reply_to': 's2383_entity_responses', 'operat
ion': 'CREATE_FIXTURE', 'data': {'fixture_name': 'HW8 Test Fixture', 'teams': ['Team A', 'Team B'], 'fixture_code': '
HW8-001', 'available_slots': 10, 'venue_id': 1}}
Client received: {'success': True, 'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'message': 'Fixture created',
 'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc'}
Generated entity_id: 1d480354-a6d8-436c-9e2e-d6ef142e6685

--- GET_FIXTURE ---
Client sent: {'correlation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc', 'reply_to': 's2383_entity_responses', 'operat
ion': 'GET_FIXTURE', 'data': {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685'}}
Client received: {'success': True, 'entity': {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'fixture_name': 'H
W8 Test Fixture', 'teams': ['Team A', 'Team B'], 'fixture_code': 'HW8-001', 'available_slots': 10, 'venue_id': 1}, 'c
orrelation_id': '3e9915bb-cc2e-4086-98d1-edcbac04bdfc'}
Retrieved fixture: {'entity_id': '1d480354-a6d8-436c-9e2e-d6ef142e6685', 'fixture_name': 'HW8 Test Fixture', 'teams':
 ['Team A', 'Team B'], 'fixture_code': 'HW8-001', 'available_slots': 10, 'venue_id': 1}
Correlation ID preserved: True

Send request and accept only matching correlation ID

```python
        try:
            # Send the request to Kafka.
            await producer.send_and_wait(
                ENTITY_REQUEST_TOPIC,
                request,
            )

            print(f"Client sent: {request}")

            # Wait for the response with the matching correlation ID.
            async for message in consumer:
                response = message.value

                if response.get("correlation_id") == correlation_id:
                    print(f"Client received: {response}")
                    return response

        finally:
            await producer.stop()
            await consumer.stop()

```

Ananya Yallapragada
019142383

Operations

kafka/worker.py - UUID creation, in-memory dictionary, CREATE/GET responses

```python
def create_fixture(data: dict) -> dict:
    error = validate_fixture(data)

    if error:
        return {
            "success": False,
            "error": error,
        }

    # Generate the entity ID in the worker as required.
    entity_id = str(uuid.uuid4())

    fixture = {
        "entity_id": entity_id,
        **data,
    }

    fixtures[entity_id] = fixture

    return {
        "success": True,
        "entity_id": entity_id,
        "message": "Fixture created",
    }


def get_fixture(data: dict) -> dict:
    entity_id = data.get("entity_id")

    if not is_non_empty_text(entity_id):
        return {
            "success": False,
            "error": "Missing required field: entity_id",
        }

    fixture = fixtures.get(entity_id)

    if fixture is None:
        return {
            "success": False,
            "error": "Fixture not found",
        }

    return {
        "success": True,
        "entity": fixture,
```

Ananya Yallapragada
019142383

Worker receives, preserves correlation ID, and replies to reply_to

```python
    async for message in consumer:
        request = message.value

        print(f"Worker received: {request}")

        correlation_id = request.get("correlation_id")
        reply_to = request.get("reply_to")

        response = process_request(request)

        # Preserve the original correlation ID in every response.
        response["correlation_id"] = correlation_id

        print(f"Worker processed: {response}")

        if reply_to:
            await producer.send_and_wait(reply_to, response)
            print(f"Worker sent response: {response}")
```

Demonstration

web_application/homework_demo.py

```python
async def main():
    # Create one correlation ID for the complete demo flow.
    correlation_id = str(uuid.uuid4())

    fixture_data = {
        "fixture_name": "HW8 Test Fixture",
        "teams": ["Team A", "Team B"],
        "fixture_code": "HW8-001",
        "available_slots": 10,
        "venue_id": 1,
    }

    print("\n--- CREATE_FIXTURE ---")

    create_response = await send_request(
        operation="CREATE_FIXTURE",
        data=fixture_data,
        correlation_id=correlation_id,
    )

    if not create_response.get("success"):
        print("CREATE_FIXTURE failed.")
        return

    # Use the generated ID returned by the worker.
    entity_id = create_response["entity_id"]

    print(f"Generated entity_id: {entity_id}")

    print("\n--- GET_FIXTURE ---")

    get_response = await send_request(
        operation="GET_FIXTURE",
        data={"entity_id": entity_id},
        correlation_id=correlation_id,
    )

    if not get_response.get("success"):
        print("GET_FIXTURE failed.")
        return

    print(f"Retrieved fixture: {get_response['entity']}")

    # Confirm that the correlation ID stayed unchanged.
    if get_response["correlation_id"] == correlation_id:
        print("Correlation ID preserved: True")
    else:
        print("Correlation ID preserved: False")
```

Ananya Yallapragada
019142383

Docker/FastAPI Evidence

web_application/kafka_api.py - Minimal FastAPI wrapper sending requests through Kafka

```python
@app.on_event("startup")
async def announce_startup() -> None:
    print("FastAPI service available at http://127.0.0.1:8583")


async def kafka_response(operation: str, data: dict) -> dict:
    correlation_id = str(uuid.uuid4())

    try:
        response = await asyncio.wait_for(
            send_request(operation, data, correlation_id),
            timeout=15,
        )
    except TimeoutError as error:
        raise HTTPException(
            status_code=504,
            detail="Timed out waiting for Kafka response",
        ) from error

    print(f"FastAPI received response: {response}")
    return response


@app.get("/")
async def root() -> dict:
    return {"message": "HW8 Kafka Fixture API"}


@app.post("/kafka/fixtures")
async def create_fixture(data: dict) -> dict:
    return await kafka_response("CREATE_FIXTURE", data)


@app.get("/kafka/fixtures/{entity_id}")
async def get_fixture(entity_id: str) -> dict:
    return await kafka_response("GET_FIXTURE", {"entity_id": entity_id})
```

Ananya Yallapragada
019142383

Part 2 - Three-Agent Kafka Pipeline

I. Topics and Workflow - covered in the subsequent sections

kafka/config.py

```python
# Part 2 pipeline topics.
INBOX_TOPIC = f"{PREFIX}_inbox"
TASKS_TOPIC = f"{PREFIX}_tasks"
DRAFTS_TOPIC = f"{PREFIX}_drafts"
FINAL_TOPIC = f"{PREFIX}_final"
```

II. Kafka Topic Creation

```
(.venv) annie@mac data260-2383 % docker exec -it kafka bash
eccc1dfc0b0d:/$
alias kt='/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092'

kt --create --topic s2383_inbox --partitions 1 --replication-factor 1

kt --create --topic s2383_tasks --partitions 2 --replication-factor 1

kt --create --topic s2383_drafts --partitions 1 --replication-factor 1

kt --create --topic s2383_final --partitions 1 --replication-factor 1
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid i
ssues it is best to use either, but not both.
Created topic s2383_inbox.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid i
ssues it is best to use either, but not both.
Created topic s2383_tasks.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid i
ssues it is best to use either, but not both.
Created topic s2383_drafts.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid i
ssues it is best to use either, but not both.
Created topic s2383_final.
eccc1dfc0b0d:/$ kt --list
__consumer_offsets
s2383_drafts
s2383_entity_requests
s2383_entity_responses
s2383_final
s2383_inbox
s2383_tasks
```

III. Running the agents and IV. Result

kafka/pipeline.py - Shared Kafka Pattern

```python
def message_consumer(topic: str, group_id: str) -> AIOKafkaConsumer:
    return AIOKafkaConsumer(
        topic,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        group_id=group_id,
        enable_auto_commit=False,
        auto_offset_reset="latest",
        value_deserializer=lambda value: json.loads(value.decode("utf-8")),
    )


def ask_model(instruction: str, content: str) -> str:
    response = complete(
        [
            {"role": "system", "content": instruction},
            {"role": "user", "content": content},
        ],
        model="qwen3:8b",
        temperature=0.0,
        max_tokens=250,
    )
    return response.content.strip()
```

Ananya Yallapragada
019142383

web_application/planner.py - Planner Agent

```
try:
    async for message in consumer:
        incoming = message.value
        print(f"Planner received: {incoming}")

        plan = await asyncio.to_thread(
            ask_model,
            "Create a short three-step plan for answering a community sport
            "league fixture question. Return plain text only.",
            incoming["question"],
        )
        task = {
            "correlation_id": incoming["correlation_id"],
            "question": incoming["question"],
            "plan": plan,
        }

        await producer.send_and_wait(TASKS_TOPIC, task)
        await consumer.commit()
        print(f"Planner sent: {task}")
finally:
    await consumer.stop()
    await producer.stop()
```

planner

```
(.venv) annie@Siris-MacBook-Pro web_application % python planner.py
Student: Ananya
Planner is ready and listening...
Planner received: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league manager
 schedule a weekday-evening tennis fixture at Spartan Tennis Complex?'}
Planner sent: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league manager sch
edule a weekday-evening tennis fixture at Spartan Tennis Complex?', 'plan': '1. Check the availability of the tennis
courts at Spartan Tennis Complex for the desired weekday evening time.  \n2. Confirm the availability of players and
coaches for the scheduled time.  \n3. Finalize the schedule and send out the fixture details to all participants and
the complex management.'}
```

Ananya Yallapragada
019142383

web_application/writer.py

```
try:
    async for message in consumer:
        incoming = message.value
        print(f"Writer received: {incoming}")

        draft = await asyncio.to_thread(
            ask_model,
            "Write a concise, practical answer for a community sports leagu
            "manager. Follow the supplied plan. Return plain text only.",
            f"Question: {incoming['question']}\n\nPlan: {incoming['plan']}"
        )
        draft_message = {
            "correlation_id": incoming["correlation_id"],
            "question": incoming["question"],
            "plan": incoming["plan"],
            "draft": draft,
        }

        await producer.send_and_wait(DRAFTS_TOPIC, draft_message)
        await consumer.commit()
        print(f"Writer sent: {draft_message}")
finally:
    await consumer.stop()
    await producer.stop()
```

writer

```
(.venv) annie@mac web_application % python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league manager
schedule a weekday-evening tennis fixture at Spartan Tennis Complex?', 'plan': '1. Check the availability of the tenn
is courts at Spartan Tennis Complex for the desired weekday evening time.  \n2. Confirm the availability of players a
nd coaches for the scheduled time.  \n3. Finalize the schedule and send out the fixture details to all participants a
nd the complex management.'}
Writer sent: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league manager sche
dule a weekday-evening tennis fixture at Spartan Tennis Complex?', 'plan': '1. Check the availability of the tennis c
ourts at Spartan Tennis Complex for the desired weekday evening time.  \n2. Confirm the availability of players and c
oaches for the scheduled time.  \n3. Finalize the schedule and send out the fixture details to all participants and t
he complex management.', 'draft': '1. Contact Spartan Tennis Complex to check court availability for the desired week
day evening time.  \n2. Reach out to players and coaches to confirm their availability for the scheduled time.  \n3.
Finalize the schedule and send out the fixture details to all participants and the complex management.'}
```

Ananya Yallapragada
019142383

web_application/reviewer.py

```
try:
    async for message in consumer:
        incoming = message.value
        print(f"Reviewer received: {incoming}")

        answer = await asyncio.to_thread(
            ask_model,
            "Review the draft for correctness, clarity, and practicality. "
            "Return an improved final answer only.",
            f"Question: {incoming['question']}\n\nDraft: {incoming['draft']}
        )
        final_message = {
            "correlation_id": incoming["correlation_id"],
            "status": "approved",
            "answer": answer,
        }

        await producer.send_and_wait(FINAL_TOPIC, final_message)
        await consumer.commit()
        print(f"Reviewer sent: {final_message}")
finally:
    await consumer.stop()
    await producer.stop()
```

reviewer

```
(.venv) annie@mac web_application % python reviewer.py
Student: Ananya
Reviewer is ready and listening...
Reviewer received: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league manage
r schedule a weekday-evening tennis fixture at Spartan Tennis Complex?', 'plan': '1. Check the availability of the te
nnis courts at Spartan Tennis Complex for the desired weekday evening time.  \n2. Confirm the availability of players
 and coaches for the scheduled time.  \n3. Finalize the schedule and send out the fixture details to all participants
 and the complex management.', 'draft': '1. Contact Spartan Tennis Complex to check court availability for the desire
d weekday evening time.  \n2. Reach out to players and coaches to confirm their availability for the scheduled time.
 \n3. Finalize the schedule and send out the fixture details to all participants and the complex management.'}
Reviewer sent: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'status': 'approved', 'answer': '1. Contact
 Spartan Tennis Complex to check court availability for the desired weekday evening time.  \n2. Coordinate with playe
rs and coaches to confirm their availability for the scheduled time.  \n3. Finalize the schedule and send out the fix
ture details to all participants and the complex management.'}
```

Ananya Yallapragada
019142383

web_application/send_question.py

```python
async def main() -> None:
    correlation_id = str(uuid.uuid4())
    consumer = message_consumer(FINAL_TOPIC, f"{PREFIX}_final_reader_{correlati
    producer = message_producer()

    await consumer.start()
    await producer.start()
    print("Student: Ananya")

    try:
        question_message = {
            "correlation_id": correlation_id,
            "question": QUESTION,
        }
        await producer.send_and_wait(INBOX_TOPIC, question_message)
        print(f"Question sender sent: {question_message}")

        async for message in consumer:
            final_message = message.value
            if final_message.get("correlation_id") == correlation_id:
                print(f"Question sender received final: {final_message}")
                await consumer.commit()
                return
    finally:
        await consumer.stop()
        await producer.stop()
```

sender

```
(.venv) annie@mac web_application % python send_question.py
Student: Ananya
Question sender sent: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'question': 'How should a league man
ager schedule a weekday-evening tennis fixture at Spartan Tennis Complex?'}
Question sender received final: {'correlation_id': '05233b10-847a-4ed9-a5a3-be03351fd65b', 'status': 'approved', 'ans
wer': '1. Contact Spartan Tennis Complex to check court availability for the desired weekday evening time.  \n2. Coor
dinate with players and coaches to confirm their availability for the scheduled time.  \n3. Finalize the schedule and
 send out the fixture details to all participants and the complex management.'}
```

Ananya Yallapragada
019142383

Part 3 - Failure Behavior

I. Duplicate Delivery and Idempotency

Before Idempotency

demo

```
(.venv) annie@mac web_application % python duplicate_delivery_demo.py --expected-drafts 2
Student: Ananya
Duplicate-delivery test correlation_id: 12b64d45-1e4a-42d2-b3c7-90ef1f61585d
Replay 1 sent: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager co
nfirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Replay 2 sent: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager co
nfirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Draft side effect 1: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league mana
ger confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft
': 'First, check with the venue and players to confirm availability. Then, send a clear message to all participants w
ith the date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.
'}
Draft side effect 2: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league mana
ger confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft
': 'First, check with the venue and players to confirm availability. Then, send a clear message to all participants w
ith the date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.
'}
Expected draft side effects observed: 2
```

Writer

```
(.venv) annie@mac web_application % IDEMPOTENCY_ENABLED=false python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer sent: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager conf
irm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft': 'Firs
t, check with the venue and players to confirm availability. Then, send a clear message to all participants with the
date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.'}
Writer received: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer sent: {'correlation_id': '12b64d45-1e4a-42d2-b3c7-90ef1f61585d', 'question': 'How should a league manager conf
irm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft': 'Firs
t, check with the venue and players to confirm availability. Then, send a clear message to all participants with the
date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.'}
```

web_application/writer.py

```
    await producer.send_and_wait(DRAFTS_TOPIC, draft_message)
    if IDEMPOTENCY_ENABLED:
        completed_correlation_ids.add(correlation_id)
    await consumer.commit()
    print(f"Writer sent: {draft_message}")
```

Ananya Yallapragada
019142383

After Idempotency

demo

```
(.venv) annie@mac web_application % python duplicate_delivery_demo.py --expected-drafts 1
Student: Ananya
Duplicate-delivery test correlation_id: 23ccf642-7d4f-410b-b11a-0fc23ff65cb9
Replay 1 sent: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league manager co
nfirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Replay 2 sent: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league manager co
nfirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Draft side effect 1: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league mana
ger confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft
': 'First, check with the venue and players to confirm availability. Then, send a clear message to all participants w
ith the date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.
'}
Expected draft side effects observed: 1
```

writer

```
(.venv) annie@mac web_application % python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer sent: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league manager conf
irm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft': 'Firs
t, check with the venue and players to confirm availability. Then, send a clear message to all participants with the
date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.'}
Writer received: {'correlation_id': '23ccf642-7d4f-410b-b11a-0fc23ff65cb9', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer duplicate skipped: 23ccf642-7d4f-410b-b11a-0fc23ff65cb9
```

II. Consumer Death

```
(.venv) annie@mac web_application % PROCESSING_DELAY_SECONDS=30 python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer processing delay: 30 seconds
```

```
(.venv) annie@mac web_application % python consumer_death_demo.py
Student: Ananya
Consumer-death test correlation_id: 8b3c54b3-300c-4d7f-a49a-23e01f920b36
Task sent before Writer restart: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How should a
 league manager confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record detai
ls.'}
```

After writer restart

```
(.venv) annie@mac web_application % PROCESSING_DELAY_SECONDS=30 python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer processing delay: 30 seconds
Writer sent: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How should a league manager conf
irm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft': 'Firs
t, check with the venue and players to confirm availability. Then, send a clear message to all participants with the
date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.'}
```

Ananya Yallapragada
019142383

```
(.venv) annie@mac web_application % python consumer_death_demo.py
Student: Ananya
Consumer-death test correlation_id: 8b3c54b3-300c-4d7f-a49a-23e01f920b36
Task sent before Writer restart: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How should a
 league manager confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record detai
ls.'}
Draft received after Writer restart: {'correlation_id': '8b3c54b3-300c-4d7f-a49a-23e01f920b36', 'question': 'How shou
ld a league manager confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record d
etails.', 'draft': 'First, check with the venue and players to confirm availability. Then, send a clear message to al
l participants with the date, time, and location. Finally, record the details in a shared calendar or spreadsheet for
 easy reference.'}
```

III. Poison Message

writer

```
(.venv) annie@mac web_application % python writer.py
Student: Ananya
Writer is ready and listening...
Writer received: {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'question': 'This task intentionally fail
s.', 'plan': 'No plan is executed.', 'force_failure': True}
Writer retry 1/3: 9aa015fc-eae8-4b64-a69d-aa36755ea05e
Writer retry 2/3: 9aa015fc-eae8-4b64-a69d-aa36755ea05e
Writer retry 3/3: 9aa015fc-eae8-4b64-a69d-aa36755ea05e
Writer sent to dead letter: {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'error': 'Forced poison-messag
e failure', 'retry_count': 3, 'original_message': {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'questio
n': 'This task intentionally fails.', 'plan': 'No plan is executed.', 'force_failure': True}}
Writer received: {'correlation_id': '8af8fda0-f676-4f0a-b8ff-3405c6650cc9', 'question': 'How should a league manager
confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.'}
Writer sent: {'correlation_id': '8af8fda0-f676-4f0a-b8ff-3405c6650cc9', 'question': 'How should a league manager conf
irm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.', 'draft': 'Firs
t, check with the venue and players to confirm availability. Then, send a clear message to all participants with the
date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy reference.'}
```

demo

```
(.venv) annie@mac web_application % python poison_message_demo.py
Student: Ananya
Poison task sent: {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'question': 'This task intentionally fai
ls.', 'plan': 'No plan is executed.', 'force_failure': True}
Valid task sent after poison task: {'correlation_id': '8af8fda0-f676-4f0a-b8ff-3405c6650cc9', 'question': 'How should
 a league manager confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record det
ails.'}
Dead-letter message received: {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'error': 'Forced poison-mess
age failure', 'retry_count': 3, 'original_message': {'correlation_id': '9aa015fc-eae8-4b64-a69d-aa36755ea05e', 'quest
ion': 'This task intentionally fails.', 'plan': 'No plan is executed.', 'force_failure': True}}
Later valid task completed: {'correlation_id': '8af8fda0-f676-4f0a-b8ff-3405c6650cc9', 'question': 'How should a leag
ue manager confirm a tennis fixture?', 'plan': '1. Confirm availability. 2. Notify participants. 3. Record details.',
 'draft': 'First, check with the venue and players to confirm availability. Then, send a clear message to all partici
pants with the date, time, and location. Finally, record the details in a shared calendar or spreadsheet for easy ref
erence.'}
```

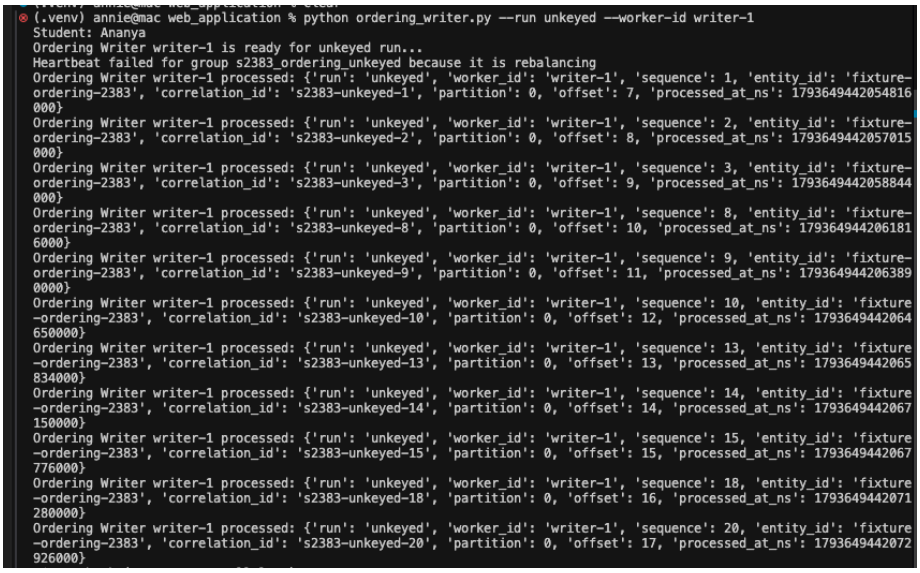
web_application/writer.py

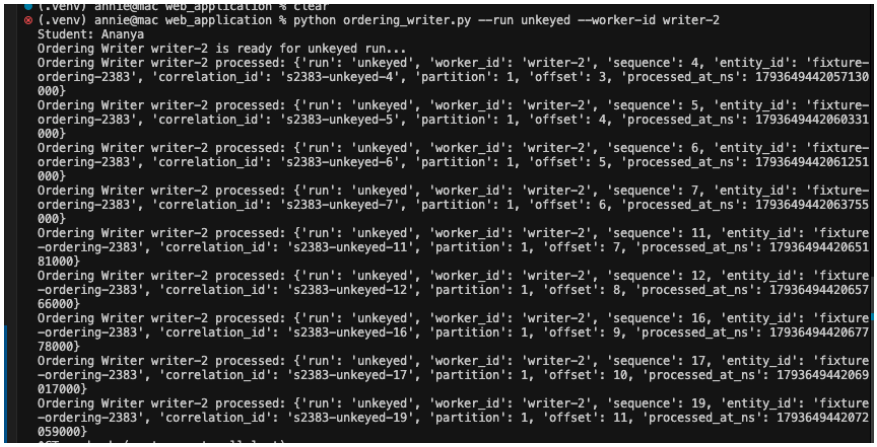
```python
    if incoming.get("force_failure"):
        for attempt in range(1, MAX_RETRIES + 1):
            print(f"Writer retry {attempt}/{MAX_RETRIES}: {correlation_
            await asyncio.sleep(1)

        dead_letter = {
            "correlation_id": correlation_id,
            "error": "Forced poison-message failure",
            "retry_count": MAX_RETRIES,
            "original_message": incoming,
        }
        await producer.send_and_wait(DEAD_LETTER_TOPIC, dead_letter)
        await consumer.commit()
        print(f"Writer sent to dead letter: {dead_letter}")
        continue
```

Ananya Yallapragada
019142383

IV. Ordering With Two Partitions and Two Consumers Unkeyed

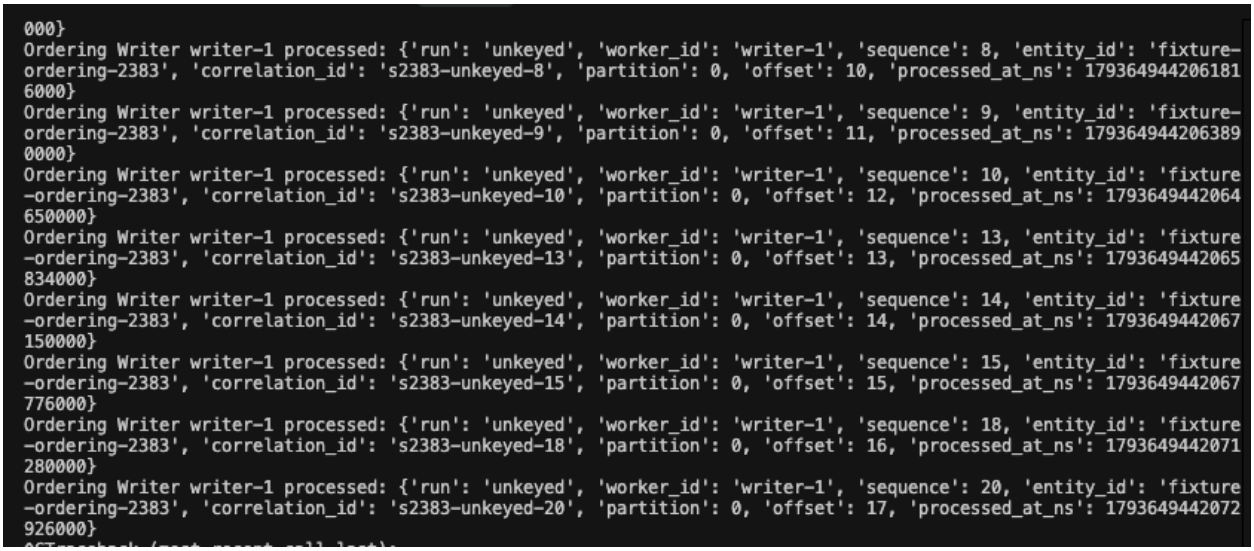


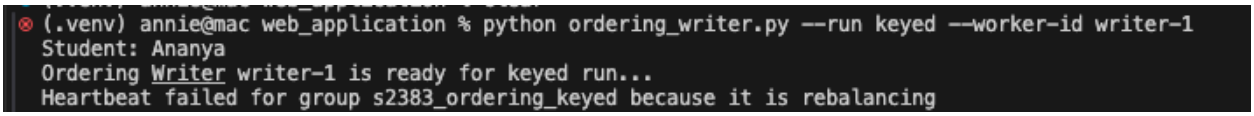




Ananya Yallapragada
019142383

Keyed run









Ananya Yallapragada
019142383

```
(.venv) annie@mac web_application % python ordering_experiment.py --run keyed
Student: Ananya
Ordering run: keyed; VERIFY_SEED=262383
Keyed message 1 sent with key fixture-ordering-2383
Keyed message 2 sent with key fixture-ordering-2383
Keyed message 3 sent with key fixture-ordering-2383
Keyed message 4 sent with key fixture-ordering-2383
Keyed message 5 sent with key fixture-ordering-2383
Keyed message 6 sent with key fixture-ordering-2383
Keyed message 7 sent with key fixture-ordering-2383
Keyed message 8 sent with key fixture-ordering-2383
Keyed message 9 sent with key fixture-ordering-2383
Keyed message 10 sent with key fixture-ordering-2383
Keyed message 11 sent with key fixture-ordering-2383
Keyed message 12 sent with key fixture-ordering-2383
Keyed message 13 sent with key fixture-ordering-2383
Keyed message 14 sent with key fixture-ordering-2383
Keyed message 15 sent with key fixture-ordering-2383
Keyed message 16 sent with key fixture-ordering-2383
Keyed message 17 sent with key fixture-ordering-2383
Keyed message 18 sent with key fixture-ordering-2383
Keyed message 19 sent with key fixture-ordering-2383
Keyed message 20 sent with key fixture-ordering-2383
(.venv) annie@mac web_application % python summarize_ordering.py --run keyed
{
  "run": "keyed",
  "message_count": 20,
  "partitions_used": [
    1
  ],
  "processing_sequence": [
    1,
    2,
    3,
    4,
    5,
    6,
    7,
    8,
    9,
    10,
    11,
    12,
    13,
    14,
    15,
    16,
    17,
    18,
    19,
    20
  ],
  "preserves_input_order": true
}
```

Ananya Yallapragada
019142383

web_application/ordering_writer.py

```
    try:
        with output_path.open("w", encoding="utf-8") as output_file:
            async for message in consumer:
                incoming = message.value
                if incoming.get("test_type") != "ordering":
                    await consumer.commit()
                    continue

                record = {
                    "run": run_name,
                    "worker_id": worker_id,
                    "sequence": incoming["sequence"],
                    "entity_id": incoming["entity_id"],
                    "correlation_id": incoming["correlation_id"],
                    "partition": message.partition,
                    "offset": message.offset,
                    "processed_at_ns": time.time_ns(),
                }
                output_file.write(json.dumps(record) + "\n")
                output_file.flush()
                await consumer.commit()
                print(f"Ordering Writer {worker_id} processed: {record}")
    finally:
        await consumer.stop()
```

web_application/summarize_ordering.py

```
sequences = [record["sequence"] for record in records]
partitions = sorted({record["partition"] for record in records})
summary = {
    "run": run_name,
    "message_count": len(records),
    "partitions_used": partitions,
    "processing_sequence": sequences,
    "preserves_input_order": sequences == list(range(1, MESSAGE_COUNT + 1))
}

json_path = RAW_DIRECTORY / f"ordering_{run_name}_summary.json"
csv_path = RAW_DIRECTORY / f"ordering_{run_name}_records.csv"
json_path.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8"

with csv_path.open("w", newline="", encoding="utf-8") as output_file:
    writer = csv.DictWriter(output_file, fieldnames=records[0].keys())
    writer.writeheader()
    writer.writerows(records)

print(json.dumps(summary, indent=2))
```

Ananya Yallapragada
019142383

V. Comparision

Part 3 — Failure Behavior

Failure scenario	Before fix	After fix	Matching evidence
Duplicate delivery	Replaying correlation ID `12b64d45-1e4a-42d2-b3c7-90ef1f61585d` produced two draft side effects.	Replaying correlation ID `23ccf642-7d4f-410b-b11a-0fc23ff65cb9` produced one draft; Writer logged `duplicate skipped`.	`RUN_LOG.txt` ; `raw/failure_results.json`
Consumer killed mid-work	Writer was stopped during a 30-second processing delay, before it sent a draft or committed the offset.	After restart, Writer received correlation ID `8b3c54b3-300c-4d7f-a49a-23e01f920b36` again and completed the draft.	`RUN_LOG.txt` ; `raw/failure_results.json`
Poison message	The baseline Writer had no bounded retry or dead-letter path for a forced failure.	Writer retried three times, sent the poison message to `s2383_dead_letter`, and then completed the later valid task.	`RUN_LOG.txt` ; `raw/failure_results.json`
Two partitions/two consumers	The unkeyed run used partitions 0 and 1; related messages completed out of order: `1, 2, 4, 3, ...`.	The keyed run used partition 1 only; all 20 related messages completed in order 1–20.	`raw/ordering_unkeyed_summary.json` ; `raw/ordering_keyed_summary.json`

Generative AI Use Disclosure

1. What I used an AI assistant for and what I did myself

I used an AI assistant to help interpret the homework requirements, inspect the existing project, suggest small Kafka code changes, and explain terminal commands. I personally started the Docker, Kafka, and Ollama services, created the required topics, ran the request/response tests, ran the Planner-Writer-Reviewer pipeline, ran the failure experiments, captured screenshots, reviewed the terminal output, updated the run log. I also used the assistant to improve the text grammar and clarity of the report.

2. One AI-produced output that was wrong/unsuitable, or one thing I independently verified

An early AI suggestion assumed that the existing MySQL Fixture database should be reused for Part 1. That was unsuitable because the HW8 instructions explicitly require the Kafka worker to store fixtures in a plain Python dictionary for this exercise.

3. How I detected the problem or verified the result

Ananya Yallapragada
019142383

I reread the instructions and compared the proposed storage approach with the requirements instead of relying on the existing HW7 application structure. I also independently verified the final worker behavior by running CREATE_FIXTURE, GET_FIXTURE, validation-error tests, the Planner-Writer-Reviewer pipeline, and the Kafka smoke test.

4. What I changed and why it works now

I changed the Part 1 worker to store fixtures in a plain Python dictionary and generate UUID entity IDs in the worker. The final implementation preserves correlation IDs, returns clean validation errors, and passes the Kafka round-trip smoke test. I also independently ran the duplicate-delivery, consumer-restart, poison-message/dead-letter, and two-partition ordering experiments. The final logs and raw files show the expected results.